

DATA STRUCTURES AND ALGORITHMS

Asymptotic Notations



The Process of Algorithm Development

- Design
 - divide&conquer, greedy, dynamic programming
- Validation
 - check whether it is correct
- Analysis
 - determine the properties of algorithm
- Implementation
- Testing
 - check whether it works for all possible cases

Analysis of Algorithm

- Analysis investigates
 - What are the properties of the algorithm?
 - in terms of time and space



Properties of an Algorithm

- Used memory space
 - Number of bits
 - Number of elements
- Running time
 - Wall-clock time in seconds
 - Number of operations
 - Number of most important operation
 - basic operation
- They are investigated as a function $T(n)$ of a parameter n indicating problem's size

Properties of an Algorithm

- The properties can be calculated
 - Emprically → after implementation
 - Theoretically → before implementation
- If there are many algorithms ideas, it is better to evaluate them without implementation
 - Exact calculation is not easy

Table Method

- *Table Method* is used to calculate complexity of an algorithm
- Example :

Adding elements of an array

<pre>sum = 0 for i = 1 to n sum = sum + a[i] report sum</pre>	steps/ exec	freq	total
	1	1	1
	2	$n+1$	$2n+2$
	2	n	$2n$
	1	1	1
			$4n+4$

6

Table Method

- Example :

Matrix addition

Assume a , b , c are $m \times n$ matrices

	steps/ exec	freq	total
for $i = 1$ to m for $j = 1$ to n $c[i,j] = a[i,j] + b[i,j]$	1	$m+1$	$m+1$
	1	$m(n+1)$	$mn+m$
	1	mn	mn
			$2mn+2m+1$

Analysis of Algorithm

- Analysis investigates
 - What are the properties of the algorithm?
 - in terms of time and space
 - How good is the algorithm ?
 - whether satisfies our needs or not
 - How it compares with others?
 - not always exact
 - Is it the best that can be done?
 - difficult !



Comparison of Algorithms

- Assume the running times of two algorithms are calculated:

For input size N

Running time of Algorithm A = $T_A(N) = 1000 N$

Running time of Algorithm B = $T_B(N) = N^2$

Which one is faster ?



Comparison of Algorithms

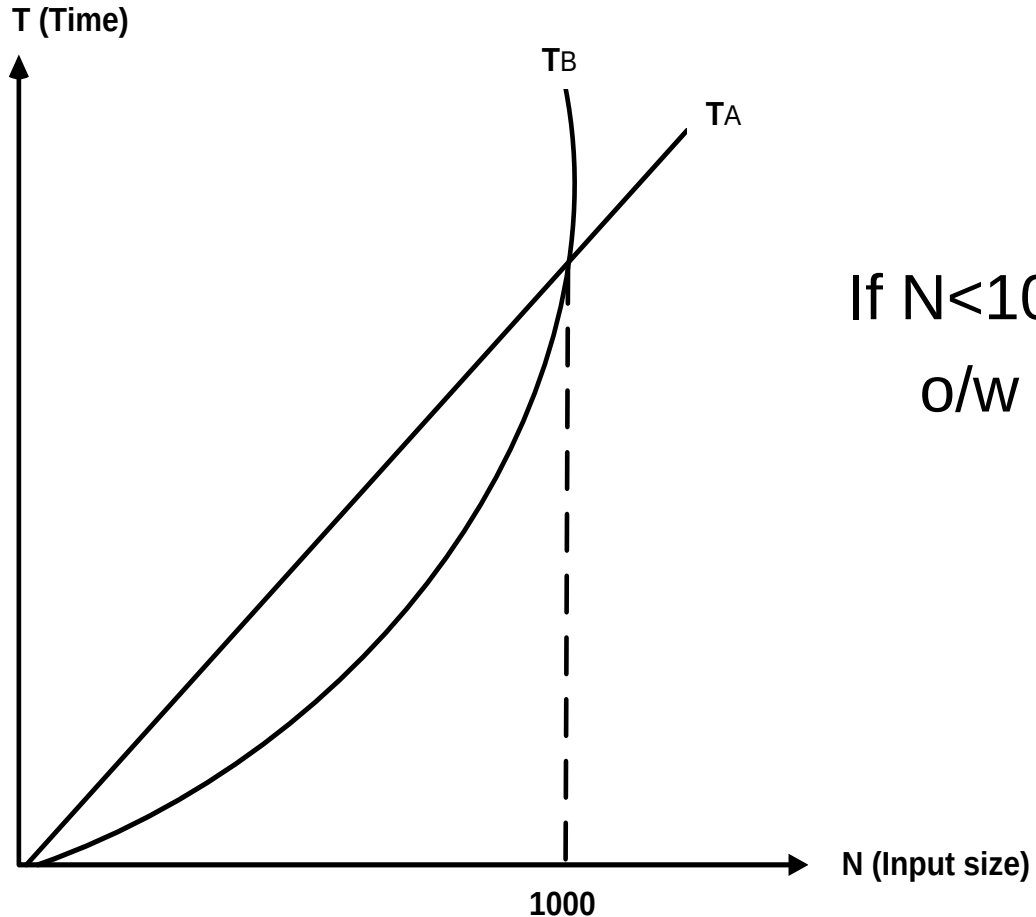
If the unit of running time of algorithms A and B is μsec

N	T_A	T_B
10	10^{-2} sec	10^{-4} sec
100	10^{-1} sec	10^{-2} sec
1000	1 sec	1 sec
10000	10 sec	100 sec
100000	100 sec	10000 sec

So which algorithm is faster ?



Comparison of Algorithms



If $N < 1000$
o/w

$$T_A(N) > T_B(N)$$

$$T_B(N) > T_A(N)$$

Compare their relative growth

Properties of an Algorithm

- Is it always possible to calculate exact value in seconds?

NO !

Run-time is effected by

- Compiler
 - O/S
 - computer
- } usually effects the constants & lower order terms
- algorithm
- } use asymtotic notation

Compare their relative growth



Comparison of Algorithms

- Is it always possible to have definite results?

NO !

The running times of algorithms can change because of the platform, the properties of the computer, etc.

We use asymptotic notations (O , Ω , θ , o)

- compare relative growth
 - No constants
 - No lower order terms
- compare only algorithms



Asymptotic Notations

- Used to express running time and memory space used.

For input size N

Running time of Alg. A = $T_A(N) = 1000 N = O(N)$

Running time of Alg. B = $T_B(N) = 7N^2 + N = O(N^2)$



Big Oh Notation (O)

Provides an “upper bound” for the function f

■ Definition :

$T(N) = O(f(N))$ if there are positive constants c and n_0 such that

$$T(N) \leq cf(N) \text{ when } N \geq n_0$$

- $T(N)$ grows no faster than $f(N)$
- growth rate of $T(N)$ is less than or equal to growth rate of $f(N)$ for large N
- $f(N)$ is an upper bound on $T(N)$
 - not fully correct !



Big Oh Notation (O)

- **Analysis of Algorithm A**

$$T_A(N) = 1000 N = O(N)$$

$$1000 N \leq cN \quad \forall N \geq n_0$$

if $c = 2000$ and $n_0 = 1$ for all N

$T_A(N) = 1000 N = O(N)$ is right



Examples

- $7n+5 = O(n)$

for $c=8$ and $n_0=5$

$$7n+5 \leq 8n \qquad n > 5 = n_0$$

- $7n+5 = O(n^2)$

for $c=7$ and $n_0=2$

$$7n+5 \leq 7n^2 \qquad n \geq n_0$$

- $7n^2+3n = O(n) \quad ?$



Advantages of O Notation

- While comparing two algorithms based on their running times
- Constants can be ignored.
 - Units are not important

$$O(7n^2) = O(n^2)$$

- Lower order terms are ignored
 - Compare relative growth only

$$O(n^3 + 7n^2 + 3) = O(n^3)$$



Big Oh Notation (O)

Running Times of Algorithm A and B

$$T_A(N) = 1000 N = O(N)$$

$$T_B(N) = N^2 = O(N^2)$$

A is asymptotically faster than B !



Omega Notation (Ω)

■ Definition :

$T(N) = \Omega(f(N))$ if there are positive constants c and n_0 such that $T(N) \geq c f(N)$ when $N \geq n_0$

- $T(N)$ grows no slower than $f(N)$
- growth rate of $T(N)$ is greater than or equal to growth rate of $f(N)$ for large N
- $f(N)$ is a lower bound on $T(N)$
 - not fully correct !



Omega Notation

Example:

- $n^{1/2} = \Omega(\lg n)$.

for $c = 1$ and $n_0 = 16$

Let $n > 16$

$$c^*(\lg n) \leq n^{1/2}$$



Omega Notation

■ Theorem:

$$f(N) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(N))$$

Proof:

$$f(N) \leq c_1 g(n) \Leftrightarrow g(n) \geq c_2 f(N)$$

divide the left side with c_1

$$1/c_1 f(N) \leq g(n) \Leftrightarrow g(n) \geq c_2 f(N)$$

if we choose c_2 as $1/c_1$ then theorem is right.

Omega Notation

- $7n^2 + 3n + 5 = O(n^4)$
- $7n^2 + 3n + 5 = O(n^3)$
- $7n^2 + 3n + 5 = O(n^2)$
- $7n^2 + 3n + 5 = \Omega(n^2)$
- $7n^2 + 3n + 5 = \Omega(n)$
- $7n^2 + 3n + 5 = \Omega(1)$

n^2 and $7n^2 + 3n + 5$ grows at the same rate

$$7n^2 + 3n + 5 = O(n^2) = \Omega(n^2) = \theta(n^2)$$



Theta Notation (θ)

■ Definition :

$T(N) = \theta(h(N))$ if and only if

$$T(N) = O(h(N)) \text{ and } T(N) = \Omega(h(N))$$

- $T(N)$ grows as fast as $h(N)$
- growth rate of $T(N)$ and $h(N)$ are equal for large N
- $h(N)$ is a tight bound on $T(N)$
 - not fully correct !



Theta Notation

- $7n^2 + 3n + 5 = O(n^4)$
- $7n^2 + 3n + 5 = O(n^3)$
- $7n^2 + 3n + 5 = O(n^2)$
- $7n^2 + 3n + 5 = \theta(n^2) \rightarrow \text{best (more information)}$
- $7n^2 + 3n + 5 = \Omega(n^2)$
- $7n^2 + 3n + 5 = \Omega(n)$
- $7n^2 + 3n + 5 = \Omega(1)$

Little o Notation (o)

■ Definition :

$T(N) = o(p(N))$ if

$T(N) = O(p(N))$ and $T(N) \neq \theta(p(N))$

- $p(N)$ grows strictly faster than $T(N)$
- growth rate of $T(N)$ is less than the growth rate of $p(N)$ for large N
- $p(N)$ is an upperbound on $T(N)$ (but not tight)
 - not fully correct !

Little o Notation (o)

■ Example :

$$T(N) = 3N^2$$

$$T(N) = o(N^4)$$

$$T(N) = o(N^3)$$

$$T(N) \neq o(N^2) \rightarrow !!!!$$

$$T(N) = \theta(N^2)$$

Some Rules

■ RULE 1:

if $T_1(N) = O(f(N))$ and $T_2(N) = O(g(N))$ then

a) $T_1(N) + T_2(N) = \max (O(f(N)), O(g(N)))$

b) $T_1(N) * T_2(N) = O(f(N) * g(N))$

You can prove these ?

Is it true for θ notation ?

What about Ω notation?



Some Rules

■ RULE 2:

if $T(N)$ is a polynomial of degree k

$$T(N) = a_k N^k + a_{k-1} N^{k-1} + \dots + a_1 N + a_0$$

then

$$T(N) = \theta(N^k)$$



Some Rules

- **RULE 3:**

$\log^k N = o(N)$ for any constant k

logarithm grows very slowly !



Some Common Functions

- $c = o(\log N) \Rightarrow c = O(\log N)$ but $c \neq \Omega(\log N)$
- $\log N = o(\log^2 N)$
- $\log^2 N = o(N)$
- $N = o(N \log N)$
- $N = o(N^2)$
- $N^2 = o(N^3)$
- $N^3 = o(2^N)$

Example

- $T(N) = 4N^2$
 - $T(N) = O(2N^2)$
correct but bad style
 $T(N) = O(N^2)$
drop the constants
 - $T(N) = O(N^2+N)$
correct but bad style
 $T(N) = O(N^2)$
ignore low order terms



Another Way to Compute Growth Rates

$$\begin{aligned} \lim_{N \rightarrow \infty} \frac{f(N)}{g(N)} = 0 & \Rightarrow f(N) = o(g(N)) \\ = c \neq 0 & \Rightarrow f(N) = \theta(g(N)) \\ = \infty & \Rightarrow g(N) = o(f(N)) \\ = \text{oscillate} & \Rightarrow \text{there is no relation} \end{aligned}$$

Example

- $f(N) = 7N^2$ $g(N) = N^2 + N$



Example

- $f(N) = N \log N$

$$g(N) = N^{1.5}$$

compare $\log N$ with $N^{0.5}$

compare $\log^2 N$ with N

compare $\log^2 N$ with $o(N)$

$$N \log N = o(N^{1.5})$$

General Rules to Analysis

■ RULE 1 : For Loops

The running time of a for loop is at most the running time of the statements in the for loop times the number of iterations

Example :

```
int i, a = 0;
for (i=0; i<n; i++)
{
    print i;
    a=a+i;
}
return i;
```

$$T(n) = \theta(n)$$



General Rules to Analysis

■ RULE 2 : Nested Loops

Analyze nested loops inside out

Example :

```
for (int i=1;i<=q;i++)  
{  
    for (int j=1;j<=r;j++)  
        k++;  
}
```

$\Theta(r)$

$\Theta(q)$

$$T(n) = \Theta(r*q)$$

General Rules to Analysis

■ **RULE 3 : Consecutive Statements**

Add the running times

for ...	}	$\theta(N)$	}	$\theta(N^2)$
...;				
for ...	}	$\theta(N^2)$		
for ...				
...;				

General Rules to Analysis

■ RULE 4 : If / Else

if (condition)	}	$T_3(n)$	}	$T(n)$
S1;	}	$T_1(n)$		
else	}	$T_2(n)$		
S2;	}			

Running time is never more than the running time of the test plus larger of the running times of S1 and S2

(may overestimate but never underestimates)

$$T(n) \leq T_3(n) + \max(T_1(n), T_2(n))$$



General Rules to Analysis

$$T_{worst}(N) = \max_{|I|=N} \{T(I)\} \rightarrow \text{usually used}$$

$$T_{av}(N) = \sum_{|I|=N} T(I) \cdot \text{Pr}(I)$$

$$T_{best}(N) = \min_{|I|=N} \{T(I)\}$$

$$T_{worst}(N) \geq T_{av}(N) \geq T_{best}(N)$$

$$T(n) = O(T_{worst}(n)) = \Omega(T_{best}(n))$$



General Rules to Analysis

■ RULE 4 : If / Else

if (condition)	}	$T_3(n)$	}	$T(n)$
S1;	}	$T_1(n)$		
else				
S2;	}	$T_2(n)$		

$$T_w(n) = T_3(n) + \max(T_1(n), T_2(n))$$

$$T_b(n) = T_3(n) + \min(T_1(n), T_2(n))$$

$$T_{av}(n) = p(T)T_1(n) + p(F)T_2(n) + T_3(n)$$

$$p(T) \rightarrow p(\text{condition} = \text{True})$$

$$p(F) \rightarrow p(\text{condition} = \text{False})$$



General Rules to Analysis

- **Example :**

if (condition)	}	$T_3(n) = \theta(n)$	}	$T(n)$
S1;	}	$T_1(n) = \theta(n^2)$		
else	}			
S2;		$T_2(n) = \theta(n)$		

$$T_w(n) = T_3(n) + \max(T_1(n), T_2(n)) = \theta(n^2)$$

$$T_b(n) = T_3(n) + \min(T_1(n), T_2(n)) = \theta(n)$$

$$\text{if } p(T) = p(F) = \frac{1}{2}$$

$$T_{av}(n) = p(T)T_1(n) + p(F)T_2(n) + T_3(n) = \theta(n^2)$$

$$\begin{aligned} T(n) &= O(n^2) \\ &= \Omega(n) \end{aligned}$$



RECURSIVE CALLS

Example:

Algorithm for computing factorial

```
int factorial (int n)
{
    if (n<=1)
        return 1;
    else
        return n*factorial(n-1);
}
```

} 1

} 1 for multiplication
+ 1 for subtraction
+ cost of evaluation of
factorial(n-1)

$T(n)$ = cost of evaluation of factorial of n

$T(n) = 2 + T(n-1)$

$T(1) = 1$



RECURSIVE CALLS

$$T(n) = 2 + T(n-1)$$

$$T(n) = 2 + 2 + T(n-2)$$

$$T(n) = 2 + 2 + 2 + T(n-3)$$

⋮

$$T(n) = k*2 + T(n-k)$$

$$k = n-1 \Rightarrow$$

$$T(n) = (n-1)*2 + T(n-(n-1))$$

$$T(n) = (n-1)*2 + T(1)$$

$$T(n) = (n-1)*2 + 1$$

$$T(n) = \theta(n)$$

